

Programming Fundamentals - 2st WEEK

Name : Dewi Novita Sari

NIM : 264107020002

Class : TI-11

JOBSHEET 2 - Variables, Data Types, Operators (Arithmetic) and Input-Output

2.1 Experiment 1: Use of Variables

Question:

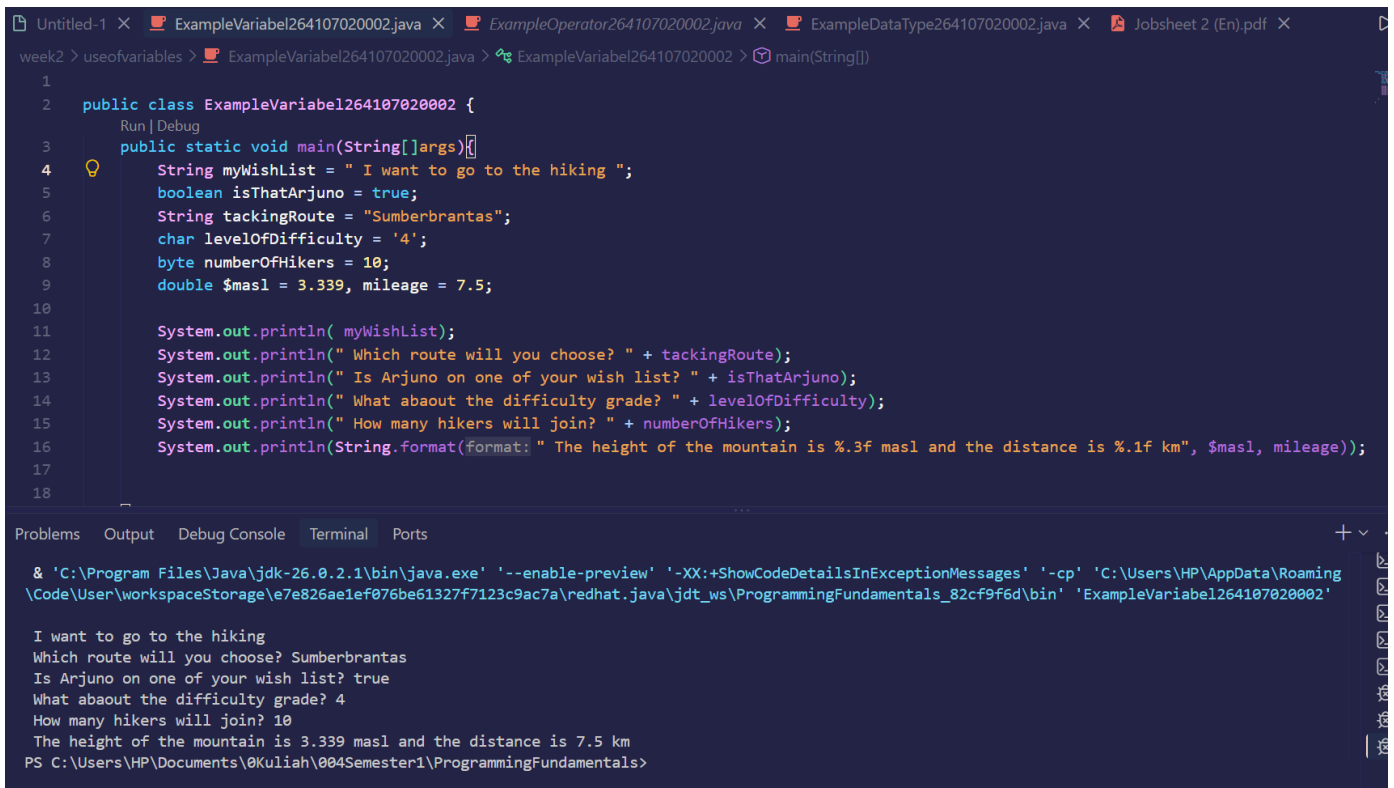
1. Change the variable name so that the variable naming model is good and correct!
2. What is the function of `%s` in the statement below?

```
System.out.println(String.format("My GPA is %s and my height is %s meters", $gpa, height)) ;
```

Is there anything you can use instead of `%s`? Name and explain it!

Answer:

1. Change the variable name

The screenshot shows an IDE with a Java file named 'ExampleVariabel264107020002.java'. The code defines a class with a main method that declares several variables: a String 'myWishList', a boolean 'isThatArjuno', a String 'tackingRoute', a char 'levelOfDifficulty', a byte 'numberOfHikers', and a double 'masl' and 'mileage'. It then uses System.out.println to print these values. The output window at the bottom shows the execution results: 'I want to go to the hiking', 'Which route will you choose? Sumberbrantas', 'Is Arjuno on one of your wish list? true', 'What abaout the difficulty grade? 4', 'How many hikers will join? 10', and 'The height of the mountain is 3.339 masl and the distance is 7.5 km'. The prompt 'PS C:\Users\HP\Documents\0Kuliah\004Semester1\ProgrammingFundamentals>' is visible at the bottom of the terminal.

2. The function of %s

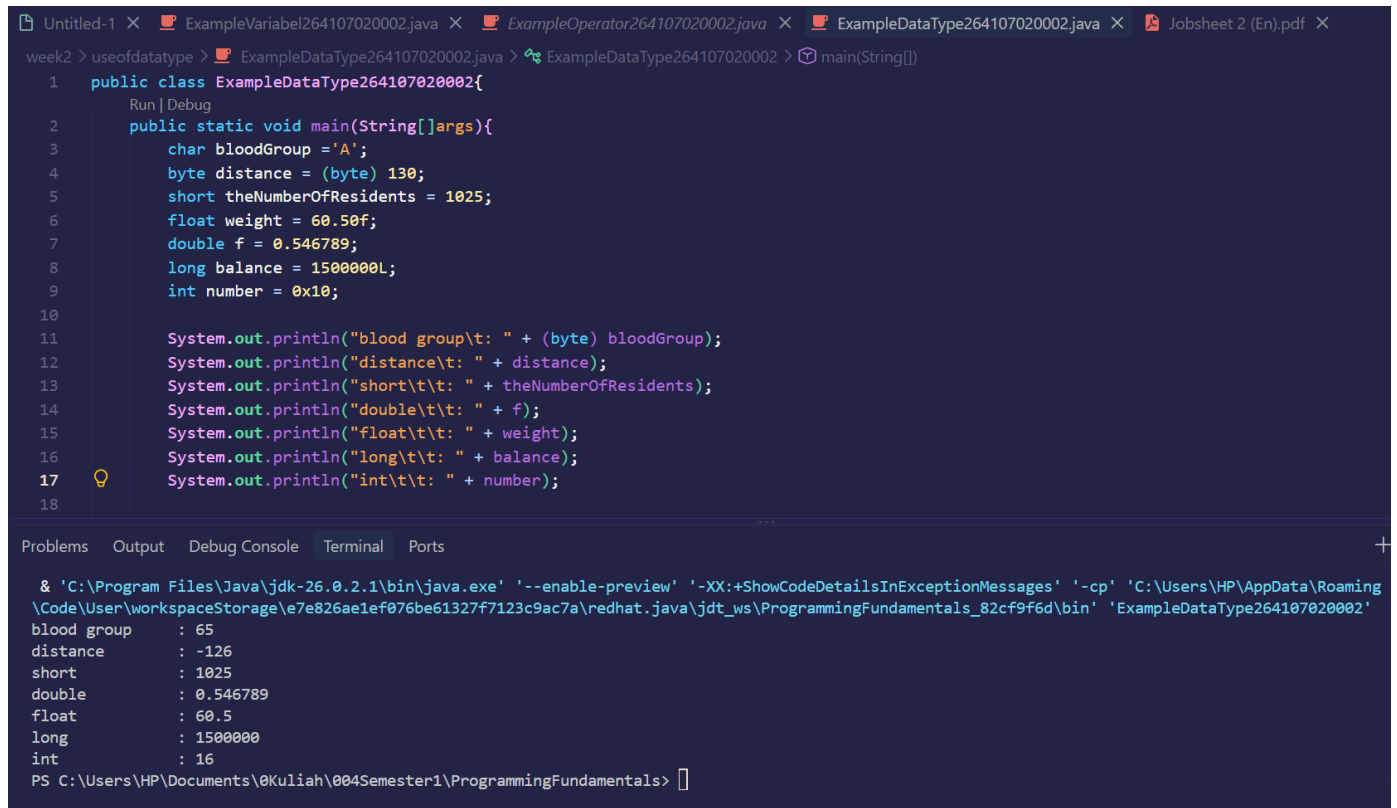
`System.out.println(String.format(format: " The height of the mountain is %.3f masl and the distance is %.1f km", $masl, mileage));`
`%s` is a placeholder for a String or text value. We can also use other format specifiers depending on the data type, such as `%d` for integers, `%f` for decimal numbers like in my code (`%.3f` allows you to specify that the number be displayed with 3 decimal places), `%b` for boolean, and `%c` for characters. We can use `println`, but `String.format()` is useful when we want to format the values, such as controlling the number of decimal places.

2.2 Experiment 2: Use of Data Types

Question:

1. Explain why the bloodGroup does not display an "A"!
2. Explain the meaning of `byte distance = (byte) 130!` Then, explain why the results change when displayed!
3. In the syntax `float temperature = 60.50F;` remove the letter F, then run again. What happened?
4. Why does the result change when displaying weight values?
5. Explain the meaning of initializing 0x10 on number variables! What does it do?

Documentation:



The screenshot shows an IDE with a Java file named `ExampleDataType264107020002.java`. The code defines a `public class ExampleDataType264107020002` with a `main` method. Inside `main`, several variables are declared and assigned values: `char bloodGroup = 'A';`, `byte distance = (byte) 130;`, `short theNumberOfResidents = 1025;`, `float weight = 60.50f;`, `double f = 0.546789;`, `long balance = 1500000L;`, and `int number = 0x10;`. Each variable is then printed to the console using `System.out.println` with a format string like `"blood group\t: " + (byte) bloodGroup;`. The output window at the bottom shows the results: `blood group : 65`, `distance : -126`, `short : 1025`, `double : 0.546789`, `float : 60.5`, `long : 1500000`, and `int : 16`. The prompt at the bottom is `PS C:\Users\HP\Documents\0Kuliah\004Semester1\ProgrammingFundamentals>`.

```
1 public class ExampleDataType264107020002{
    Run | Debug
2     public static void main(String[] args){
3         char bloodGroup = 'A';
4         byte distance = (byte) 130;
5         short theNumberOfResidents = 1025;
6         float weight = 60.50f;
7         double f = 0.546789;
8         long balance = 1500000L;
9         int number = 0x10;
10
11         System.out.println("blood group\t: " + (byte) bloodGroup);
12         System.out.println("distance\t: " + distance);
13         System.out.println("short\t\t: " + theNumberOfResidents);
14         System.out.println("double\t\t: " + f);
15         System.out.println("float\t\t: " + weight);
16         System.out.println("long\t\t: " + balance);
17         System.out.println("int\t\t: " + number);
18     }
19 }
```

Problems Output Debug Console Terminal Ports

```
& 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\e7e826ae1ef076be61327f7123c9ac7a\redhat.java\jdt_ws\ProgrammingFundamentals_82cf9f6d\bin' 'ExampleDataType264107020002'
blood group : 65
distance : -126
short : 1025
double : 0.546789
float : 60.5
long : 1500000
int : 16
PS C:\Users\HP\Documents\0Kuliah\004Semester1\ProgrammingFundamentals>
```

Answer:

1. Why does `bloodGroup` not display an "A"

Because `bloodGroup` is stored as a `char`, Java treats it as a character, not as a `String`. In ASCII (stands for American Standard Code for Information Interchange), the character `'A'` is represented by the number `65`. Therefore, when Java treats `bloodGroup` as a number, it displays `65` instead of `"A"`.

2. The meaning of `byte distance = (byte) 130!` Then, explain why the results change when displayed

`byte distance = (byte) 130;` means that the value `130` is explicitly converted to the `byte` data type. Since a `byte` can only store values from `-128` to `127`, `130` is outside the allowed range. Therefore, Java converts it using type casting, causing the displayed result to become `-126`.

3. Syntax `float temperature = 60.50F;` remove the letter `F`, then run again

```
1 public class ExampleDataType264107020002{
    Run | Debug
2     public static void main(String[] args){
3         char bloodGroup = 'A';
4         byte distance = (byte) 130;
5         short theNumberOfResidents = 1025;
6         float weight = 60.50;
7         double f = 0.546789;
8         long balance = 1500000L;
9         int number = 0x10;
10
11         System.out.println("blood group\t: " + (byte) bloodGroup);
12         System.out.println("distance\t: " + distance);
13         System.out.println("short\t\t: " + theNumberOfResidents);
14         System.out.println("double\t\t: " + f);
15         System.out.println("float\t\t: " + weight);
16         System.out.println("long\t\t: " + balance);
17         System.out.println("int\t\t: " + number);
18
19     }
20 }
```

Problems 1 Output Debug Console Terminal Ports

```
PS C:\Users\HP\Documents\0Kuliah\004Semester1\ProgrammingFundamentals>
PS C:\Users\HP\Documents\0Kuliah\004Semester1\ProgrammingFundamentals> c:: cd 'c:\Users\HP\Documents\0Kuliah\004Semester1\ProgrammingFundamentals'; & 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetails' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\e7e826ae1ef076be61327f7123c9ac7a\redhat...' '_ws\ProgrammingFundamentals_82cf9f6d\bin' 'ExampleDataType264107020002'
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
    Type mismatch: cannot convert from double to float
    at ExampleDataType264107020002.main(ExampleDataType264107020002.java:6)
```

When the letter **F** is removed, the program gives an **error** because Java treats **60.50** as a **double** by default. The **F** is needed to tell Java that the number is a **float**.

4. Why does the result change when displaying weight values?

The result changes because different data types store and display numbers differently. When a decimal weight value is converted to another data type, such as **int**, the decimal part may be removed or the value may be represented differently.

5. The meaning of initializing 0x10 on number variables

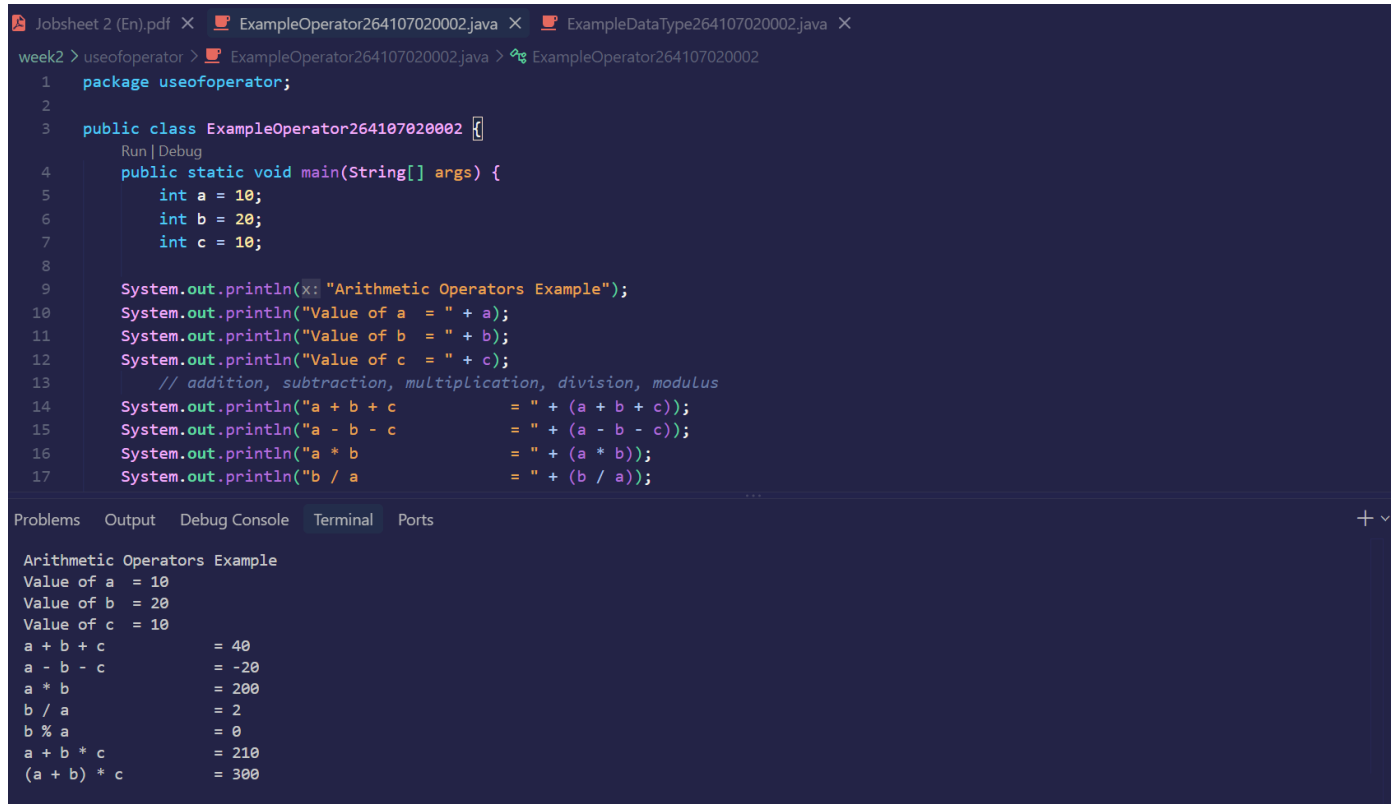
Initializing **0x10** means assigning the hexadecimal value **0x10** to a number variable. The **0x** tells Java that the number is written in hexadecimal (base 16). The value **0x10** is equal to 16 in decimal, so the variable will store the value 16. It is simply another way of writing a number.

2.3 Experiment 3: Use of Operators

Question:

1. Explain, in your opinion, the difference between **a + b * c** and **(a + b) * c**.
2. What is the difference between **a / b** and **a % b**?

Documentation:



```
Jobsheet 2 (En).pdf X ExampleOperator264107020002.java X ExampleDataType264107020002.java X
week2 > useoperator > ExampleOperator264107020002.java > ExampleOperator264107020002
1 package useoperator;
2
3 public class ExampleOperator264107020002 {
4     Run | Debug
5     public static void main(String[] args) {
6         int a = 10;
7         int b = 20;
8         int c = 10;
9
10        System.out.println("Arithmetic Operators Example");
11        System.out.println("Value of a = " + a);
12        System.out.println("Value of b = " + b);
13        System.out.println("Value of c = " + c);
14        // addition, subtraction, multiplication, division, modulus
15        System.out.println("a + b + c = " + (a + b + c));
16        System.out.println("a - b - c = " + (a - b - c));
17        System.out.println("a * b = " + (a * b));
18        System.out.println("b / a = " + (b / a));
19    }
20 }
```

Problems Output Debug Console Terminal Ports

```
Arithmetic Operators Example
Value of a = 10
Value of b = 20
Value of c = 10
a + b + c = 40
a - b - c = -20
a * b = 200
b / a = 2
b % a = 0
a + b * c = 210
(a + b) * c = 300
```

Answer:

1. Difference between $a + b * c$ and $(a + b) * c$

The difference is the order in which the calculation is performed. In $a + b * c$, multiplication is done first because multiplication has a higher priority than addition.

So:

$$a + b * c = 10 + (20 \times 10) = 210$$

Meanwhile, $(a + b) * c$ calculates the addition first because the brackets have priority:

$$(a + b) * c = (10 + 20) \times 10 = 300$$

In simple words, the brackets change the order of the calculation, so the results are different.

2. Difference between a / b and $a \% b$

$/$ is used for division, while $\%$ is used to find the remainder after division.

For example:

$$a / b = 10 / 20 = 0$$

Because a and b are integers int , Java gives the whole-number result and ignores the decimal part.

While:

$$a \% b = 10 \% 20 = 10$$

because 10 cannot be divided by 20 completely, so the remainder is 10.

In simple words, `/` gives the result of division, while `%` gives the remainder of the division.

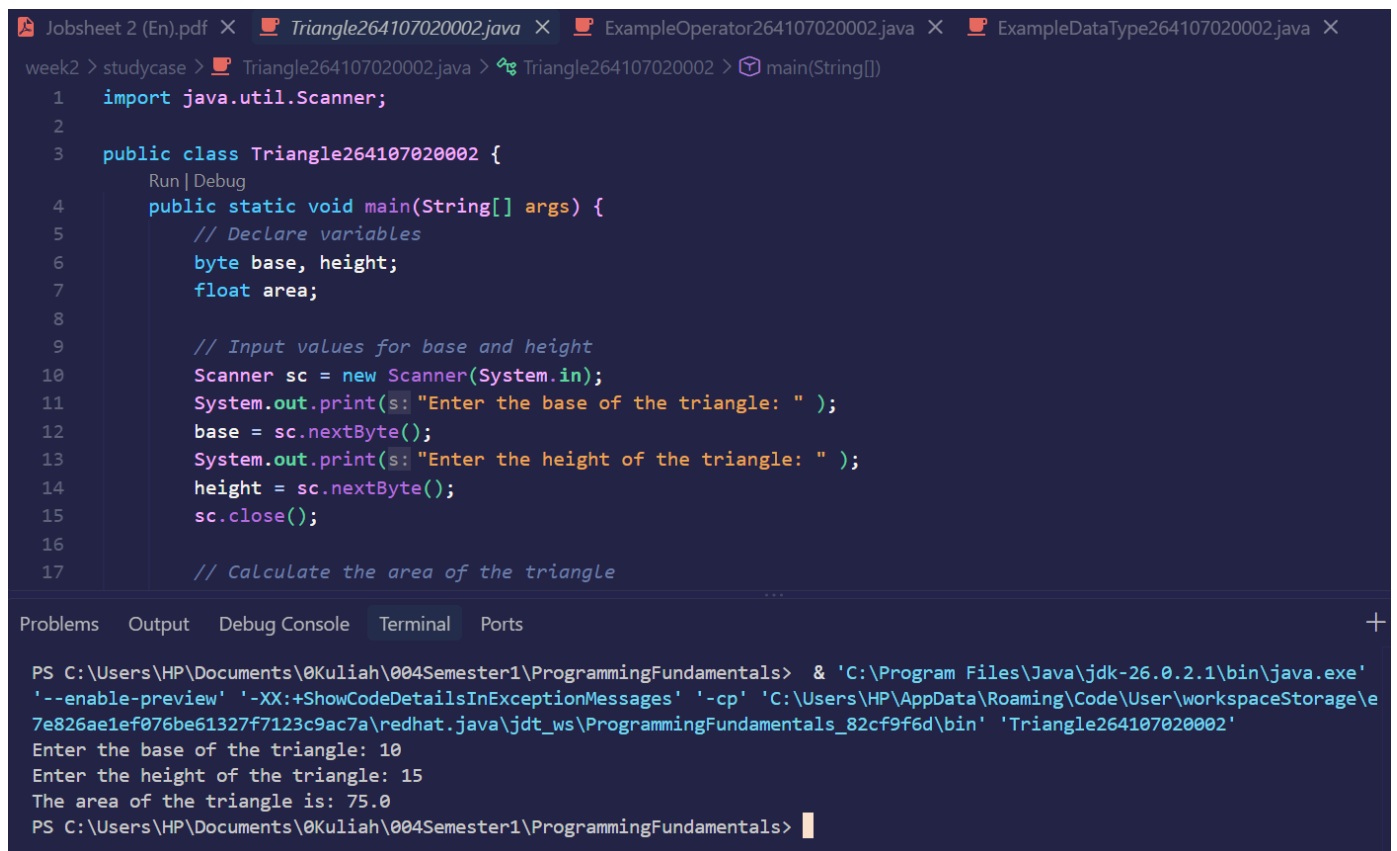
2.4 Experiment 4: Case Study

Question:

1. Explain why you have to declare `Scanner` in Experiment 4?
2. Explain the use of the program snippets below!

```
base=sc.nextInt();  
height = sc.nextInt();
```

Documentation:



The screenshot shows an IDE with several tabs open: 'Jobsheet 2 (En).pdf', 'Triangle264107020002.java', 'ExampleOperator264107020002.java', and 'ExampleDataType264107020002.java'. The active file is 'Triangle264107020002.java'. The code in the editor is as follows:

```
1 import java.util.Scanner;  
2  
3 public class Triangle264107020002 {  
4     public static void main(String[] args) {  
5         // Declare variables  
6         byte base, height;  
7         float area;  
8  
9         // Input values for base and height  
10        Scanner sc = new Scanner(System.in);  
11        System.out.print(s: "Enter the base of the triangle: " );  
12        base = sc.nextByte();  
13        System.out.print(s: "Enter the height of the triangle: " );  
14        height = sc.nextByte();  
15        sc.close();  
16  
17        // Calculate the area of the triangle
```

Below the code editor, the 'Terminal' tab is active, showing the command prompt output:

```
PS C:\Users\HP\Documents\0Kuliah\004Semester1\ProgrammingFundamentals> & 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe'  
'--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\e  
7e826ae1ef076be61327f7123c9ac7a\redhat.java\jdt_ws\ProgrammingFundamentals_82cf9f6d\bin' 'Triangle264107020002'  
Enter the base of the triangle: 10  
Enter the height of the triangle: 15  
The area of the triangle is: 75.0  
PS C:\Users\HP\Documents\0Kuliah\004Semester1\ProgrammingFundamentals>
```

Answer:

1. Why do you have to declare Scanner in Experiment 4?

We need to declare `Scanner` because the program needs to receive input from the user through the keyboard.

In the program, this line:

```
Scanner sc = new Scanner(System.in);
```

creates a `Scanner` object called `sc`. The `Scanner` allows the program to read the values that the user enters, such as the base and height of the triangle.

2. Explain the use of these program snippets

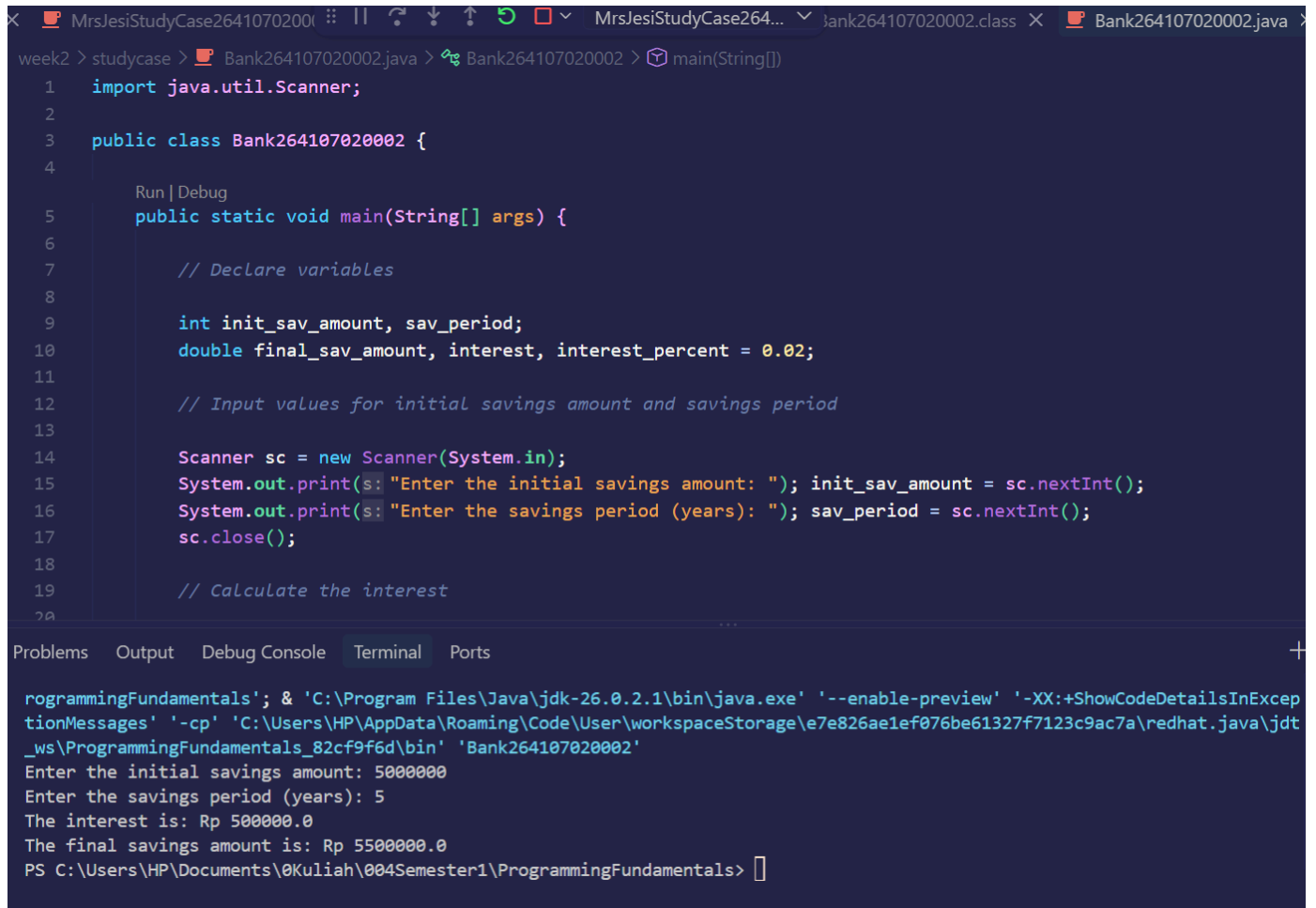
`base = sc.nextInt();` reads an integer entered by the user and stores it in the `base` variable. `height = sc.nextInt();` does the same thing for the height variable. These commands allow the program to get the base and height values from the user.

2.5 Experiment 5: Case Study

Question:

Run the program code that you have written, then observe the results

Documentation:



```
week2 > studycase > Bank264107020002.java > Bank264107020002 > main(String[])
1  import java.util.Scanner;
2
3  public class Bank264107020002 {
4
5      Run | Debug
6      public static void main(String[] args) {
7
8          // Declare variables
9
10         int init_sav_amount, sav_period;
11         double final_sav_amount, interest, interest_percent = 0.02;
12
13         // Input values for initial savings amount and savings period
14
15         Scanner sc = new Scanner(System.in);
16         System.out.print(s: "Enter the initial savings amount: "); init_sav_amount = sc.nextInt();
17         System.out.print(s: "Enter the savings period (years): "); sav_period = sc.nextInt();
18         sc.close();
19
20         // Calculate the interest
21     }
```

Problems Output Debug Console Terminal Ports

```
rogrammingFundamentals'; & 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\e7e826ae1ef076be61327f7123c9ac7a\redhat.java\jdt_ws\ProgrammingFundamentals_82cf9f6d\bin' 'Bank264107020002'
Enter the initial savings amount: 5000000
Enter the savings period (years): 5
The interest is: Rp 500000.0
The final savings amount is: Rp 5500000.0
PS C:\Users\HP\Documents\0Kuliah\004Semester1\ProgrammingFundamentals> |
```

Answer:

The program uses `Scanner` to receive the initial savings amount and savings period from the user. The `int` data type is used for the initial savings amount and savings period because both values are entered as whole numbers. The `double` data type is used for the interest, interest percentage, and final savings amount because the calculation can produce decimal values. The value `0.02` represents the 2% annual interest rate provided by the bank, while `nextInt()` is used to read the integer values entered by the user. After the user enters the required values, the program calculates the interest by multiplying the savings period, interest rate, and initial savings amount. Finally, the interest is added to the initial savings amount to calculate the final savings amount.

3 Assignment

Question:

1. Case Study 1:

Mrs. Jesi is an employee at PT. ABCD with a basic salary of Rp. 3,000,000 per month. At PT. ABCD, every employee receives a child allowance based on the number of children they have. The allowance is Rp. 150,000 per child per month. In addition, employees are required to contribute 5% of their basic salary each month to a mandatory pension fund.

Task: Write a program to calculate Mrs. Jesi's net monthly salary if she has 3 children.

2. Modify the program from Case Study 1

by making the basic salary, child allowance, and number of children dynamic inputs.

3. Case Study 2:

Mr. Jaka owns a piece of land with a width of 50 meters and a length of 100 meters. He plans to build two fish ponds: one circular pond with a diameter of 2 meters and one square pond with a side length of 2 meters. The remaining land will be planted with garden grass.

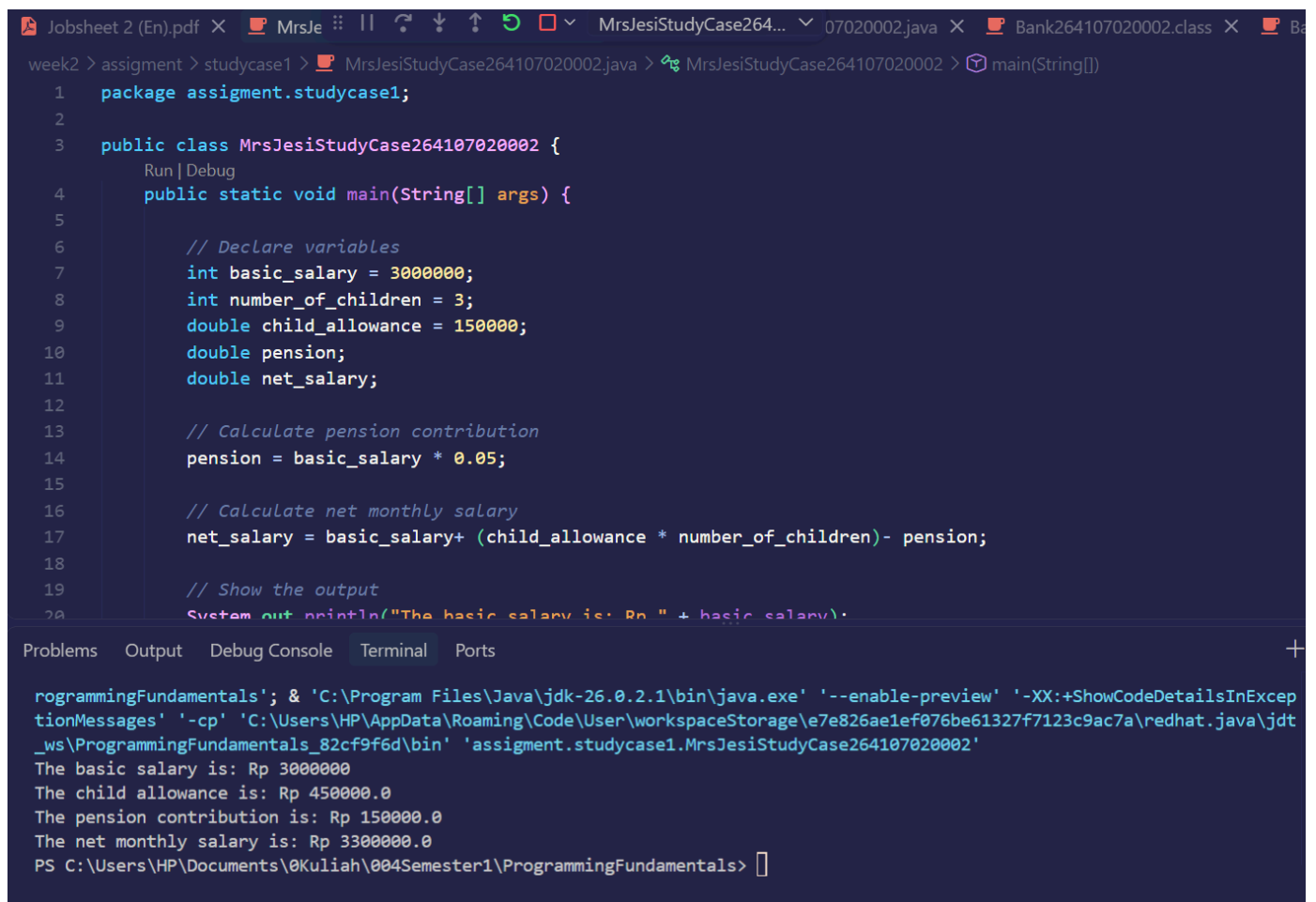
Task: Write a program to calculate the total area of land that will be planted with grass.

4. Modify the program from Case Study 2

by making the width, length, diameter, and side length dynamic inputs.

Answer:

1. Case Study 1:



```
1 package assignment.studycase1;
2
3 public class MrsJesiStudyCase264107020002 {
4     public static void main(String[] args) {
5
6         // Declare variables
7         int basic_salary = 3000000;
8         int number_of_children = 3;
9         double child_allowance = 150000;
10        double pension;
11        double net_salary;
12
13        // Calculate pension contribution
14        pension = basic_salary * 0.05;
15
16        // Calculate net monthly salary
17        net_salary = basic_salary+ (child_allowance * number_of_children)- pension;
18
19        // Show the output
20        System.out.println("The basic salary is: Rp " + basic_salary);
```

Problems Output Debug Console Terminal Ports

```
rogrammingFundamentals'; & 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\e7e826ae1ef076be61327f7123c9ac7a\redhat.java\jdt_ws\ProgrammingFundamentals_82cf9f6d\bin' 'assignment.studycase1.MrsJesiStudyCase264107020002'
The basic salary is: Rp 3000000
The child allowance is: Rp 450000.0
The pension contribution is: Rp 150000.0
The net monthly salary is: Rp 3300000.0
PS C:\Users\HP\Documents\0Kuliah\004Semester1\ProgrammingFundamentals> |
```

The program uses the basic salary, child allowance, and number of children provided in the case study. First, it calculates the child allowance by multiplying the number of children by Rp150,000. Then, the pension contribution is calculated as 5% of the basic salary. Finally, the net monthly salary is calculated by adding the child allowance to the basic salary and subtracting the pension contribution.

2.The program is modified by making the basic salary

```
week2 > assigment > studycase1 > MrsJesiStudyCase264107020002.java > MrsJesiStudyCase264107020002 > main(String[])
3
4  blic class MrsJesiStudyCase264107020002 {
5
6      Run | Debug
7      public static void main(String[] args) {
8
9          int basic_salary, number_of_children;
10         double child_allowance_per_child, child_allowance, pension, net_salary;
11
12         Scanner sc = new Scanner(System.in);
13
14         System.out.print(s: "Enter the basic salary: "); basic_salary = sc.nextInt();
15         System.out.print(s: "Enter the child allowance per child: "); child_allowance_per_child = sc.nextDouble();
16         System.out.print(s: "Enter the number of children: "); number_of_children = sc.nextInt();
17
18         pension = basic_salary * 0.05;
19         child_allowance = child_allowance_per_child * number_of_children;
20         net_salary = basic_salary + child_allowance - pension;
21
22         System.out.println("The child allowance is: Rp " + child_allowance);
23         System.out.println("The pension contribution is: Rp " + pension);
24         System.out.println("The net monthly salary is: Rp " + net_salary);
25     }
26 }
```

Problems Output Debug Console Terminal Ports

```
Enter the basic salary: 3000000
Enter the child allowance per child: 150000
Enter the number of children: 3
The child allowance is: Rp 450000.0
The pension contribution is: Rp 150000.0
The net monthly salary is: Rp 3300000.0
```

The program is modified by making the basic salary, child allowance per child, and number of children as dynamic inputs. The user can enter different values when the program runs, and the program will calculate the child allowance, pension contribution, and net monthly salary based on the given inputs.

3. Study Case 2

```
3
4 public class MrJakaStudyCase264107020002 {
5
6     Run | Debug
7     public static void main(String[] args) {
8
9         // Declare variables
10
11         double land_width, land_length;
12         double diameter, side;
13         double land_area, circle_area, square_area, grass_area;
14         double pi = 3.14;
15
16         // Input values
17
18         Scanner sc = new Scanner(System.in);
19
20         System.out.print(s: "Enter the width of the land: "); land_width = sc.nextDouble();
21         System.out.print(s: "Enter the length of the land: "); land_length = sc.nextDouble();
22     }
23 }
```

Problems Output Debug Console Terminal Ports

```
ws\ProgrammingFundamentals_82cf9f6d\bin' 'assignment.studyCase1.MrJakaStudyCase264107020002'
Enter the width of the land: 50
Enter the length of the land: 100
Enter the diameter of the circular pond: 2
Enter the side length of the square pond: 2
The total area of land is: 5000.0 m2
The area of the circular pond is: 3.14 m2
The area of the square pond is: 4.0 m2
The area planted with grass is: 4992.86 m2
PS C:\Users\HP\Documents\0Kuliah\004Semester1\ProgrammingFundamentals>
```

The program successfully calculates the total area of land planted with grass. Based on the input values, the total land area is 5,000 m², while the circular and square ponds use 3.14 m² and 4 m². Therefore, the remaining area for garden grass is 4,992.86 m².

4. Modify the program from Case Study 2

```
1 package assignment.studyCase1;
2 import java.util.Scanner;
3
4 public class MrJakaStudyCase264107020002 {
5     Run | Debug
6     public static void main(String[] args) {
7
8         // Declare variables
9
10        double land_width, land_length;
11        double diameter, side;
12        double land_area, circle_area, square_area, grass_area;
13        double pi = 3.14;
14
15        // Input values
16
17        Scanner sc = new Scanner(System.in);
18
19        System.out.print(s: "Enter the width of the land: ");
20        land_width = sc.nextDouble();
21
22        System.out.print(s: "Enter the length of the land: ");
23        land_length = sc.nextDouble();
24    }
25 }
```

Problems Output Debug Console Terminal Ports

```
Enter the width of the land: 50
Enter the length of the land: 100
Enter the diameter of the circular pond: 2
Enter the side length of the square pond: 2
The total area of land is: 5000.0 m2
The area of the circular pond is: 3.14 m2
The area of the square pond is: 4.0 m2
The area planted with grass is: 4992.86 m2
```

The program was modified by making the width, length, diameter, and side length as dynamic inputs. This allows the user to enter different values when the program runs. The program then calculates the total land area, the area of each pond, and the remaining area that can be planted with grass based on the input values.